

```

import numpy as np

# --- PART A: Perceptron ---
class Perceptron:
    def __init__(self, input_size, lr=0.1, epochs=10):
        self.weights = np.zeros(input_size + 1) # Includes bias
        self.lr = lr

```

```

        self.epochs = epochs

    def activate(self, x):
        return 1 if x >= 0 else 0

    def train(self, X, y):
        for _ in range(self.epochs):
            for xi, target in zip(X, y):
                inputs = np.insert(xi, 0, 1) # Insert bias unit
                prediction = self.activate(np.dot(inputs, self.weights))
                self.weights += self.lr * (target - prediction) * inputs

    def predict(self, X):
        inputs = np.insert(X, 0, 1)
        return self.activate(np.dot(inputs, self.weights))

# --- PART B: Backpropagation ---
def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x): return x * (1 - x)

def train_backpropagation(X, y, epochs=5000, lr=0.1):
    input_layer_size = X.shape[1]
    hidden_layer_size = 3
    output_layer_size = 1

    wh = np.random.uniform(size=(input_layer_size, hidden_layer_size))
    wout = np.random.uniform(size=(hidden_layer_size, output_layer_size))

    for _ in range(epochs):
        # Forward pass
        hidden_layer_input = np.dot(X, wh)
        hidden_layer_activations = sigmoid(hidden_layer_input)
        output_layer_input = np.dot(hidden_layer_activations, wout)
        predicted_output = sigmoid(output_layer_input)

        # Backpropagation
        error = y - predicted_output
        d_predicted_output = error * sigmoid_derivative(predicted_output)

        hidden_layer_error = d_predicted_output.dot(wout.T)
        d_hidden_layer = hidden_layer_error * sigmoid_derivative(hidden_layer_activations)

        wout += hidden_layer_activations.T.dot(d_predicted_output) * lr
        wh += X.T.dot(d_hidden_layer) * lr

    return predicted_output

# Example Usage

```

Program 8: Perceptron and Backpropagation Neural Network

```
--- Training Perceptron (AND Gate) ---
```

```
Prediction for [1, 1]: 1
```

```
--- Training Backpropagation (XOR Gate) ---
```

```
Final outputs after 5000 epochs:
```

```
[[0.44693893]  
 [0.50183621]  
 [0.57727151]  
 [0.53496814] ]
```

```
Shivani kumari 24becs105
```